

CSE422: Artificial Intelligence Lab

Project Report

Cricket Player Role Prediction

Classification of Player Position from Biographical and Playing-Style Attributes

Dataset: Cricket Players Dataset

ID	Name
24101339	Kazi Zarif Yamin
23101363	Syeda Fatima Tasnia

Submitted for CSE422 Lab Project

Table of Contents

1. Introduction.....	3
2. Dataset Description.....	3
2.1 Number of features	3
2.2 Classification or regression?.....	3
2.3 Number of data points	3
2.4 Feature types — quantitative vs. categorical	3
2.5 Do we need to encode the categorical variables?	4
2.6 Correlation heatmap	4
2.7 Imbalanced dataset	4
2.8 Exploratory data analysis — important relationships	5
3. Dataset Pre-processing	7
3.1 Problem: missing categorical values	7
3.2 Problem: the 25 unparseable dateofbirth values	8
3.3 Problem: the placeholder DOB pattern.....	8
3.4 Problem: engineering a usable age feature	8
3.5 Problem: nominal categorical variables need numeric encoding.....	9
3.6 Problem: numeric feature scale	9
3.7 Problem: irrelevant / identifier columns	9
3.8 Problem: class imbalance	9
3.9 Problem: teams.csv adds little value.....	9
4. Dataset Splitting	9
5. Model Training & Testing (Supervised)	9
6. Model Selection / Comparison Analysis.....	10
6.1 Accuracy comparison	10
6.2 Macro F1 comparison (imbalance-aware).....	10
6.3 Full comparison table	11
6.4 Confusion matrices and ROC curves.....	11
6.5 Interpretation of results.....	12
6.6 Confusion matrix discussion	13
6.7 Cross-validation.....	13
6.8 K-Means clustering (unsupervised learning)	13
7. Conclusion	14

1. Introduction

This project applies supervised and unsupervised machine learning techniques to a real-world cricket player dataset in order to predict a player's on-field role. A player's role — Batsman, Bowler, Allrounder, or Wicketkeeper — is normally assigned by cricket boards and scouts based on a player's skills and match history. This project explores whether that role can instead be inferred purely from a player's recorded biographical and playing-style attributes: batting style, bowling style, continent of origin, gender, and age.

The motivation for this project is twofold. First, it is a genuine multi-class classification problem drawn from real, messy, publicly-sourced sports data rather than a cleaned textbook dataset, which makes data-quality investigation itself a meaningful part of the exercise. Second, role classification has practical relevance in cricket analytics and scouting pipelines, where quickly categorizing large numbers of player profiles by role is a real, recurring task.

The project follows the complete standard machine learning workflow: dataset inspection, exploratory data analysis, data cleaning and preprocessing, train/test splitting, training multiple supervised classifiers (Logistic Regression, K-Nearest Neighbors, Decision Tree, Naive Bayes, a Neural Network, and Random Forest as an additional comparison model), evaluating and comparing them with imbalance-aware metrics, validating with cross-validation, and finally applying K-Means as a separate unsupervised exploration of the same feature space. Every result reported in this document was produced by actually executing the accompanying notebook (CSE422_Project_PlayerRole.ipynb) on the real dataset — no numbers in this report are invented or estimated.

2. Dataset Description

The primary dataset (players_data_with_all_info.csv) contains 17,385 player records across 16 original columns, sourced from a cricket statistics API. A companion file, teams.csv (78 rows), maps country IDs to national team names.

2.1 Number of features

After dropping identifier, URL, and timestamp columns (id, firstname, lastname, fullname, image_path, country_image_path, updated_at, country_id, continent_id, and the raw dateofbirth string, which is replaced by engineered features), the final feature set used for modeling consists of 6 features: battingstyle, bowlingstyle, continent_name, gender, age (engineered), and is_dob_estimated (engineered data-quality flag).

2.2 Classification or regression?

This is a classification problem, not a regression problem. The target variable, player position, is a categorical label (Batsman / Bowler / Allrounder / Wicketkeeper) with no inherent numeric ordering, so the goal is to assign each player to one of a finite set of discrete classes rather than to predict a continuous numeric quantity.

2.3 Number of data points

17,385 player records in total. After the train/test split described in Section 4, 13,908 records (80%) form the training set and 3,477 records (20%) form the held-out test set.

2.4 Feature types — quantitative vs. categorical

Feature	Type	Notes
battingstyle	Categorical (nominal)	42.1% missing in the raw data
bowlingstyle	Categorical (nominal)	54.7% missing in the raw data
continent_name	Categorical (nominal)	6 unique values
gender	Categorical (nominal)	94% male, 6% female
age	Quantitative (continuous)	Engineered from dateofbirth

Feature	Type	Notes
is_dob_estimated	Binary flag	Engineered data-quality indicator

2.5 Do we need to encode the categorical variables?

Yes. All machine learning algorithms used in this project (Logistic Regression, KNN, Decision Tree, Naive Bayes, Neural Network, Random Forest) require numeric input. Since `battingstyle`, `bowlingstyle`, `continent_name`, and `gender` are nominal categories with no natural order, one-hot encoding is used instead of label/ordinal encoding — assigning arbitrary integer codes (e.g. 0, 1, 2, ...) to nominal categories would falsely imply a numeric ordering or distance relationship that several of our models (KNN, Logistic Regression, Neural Network) would incorrectly use during training.

2.6 Correlation heatmap

For exploratory purposes only, the categorical features were label-encoded (not one-hot encoded) purely to allow a numeric correlation matrix to be computed with the seaborn heatmap function shown below.

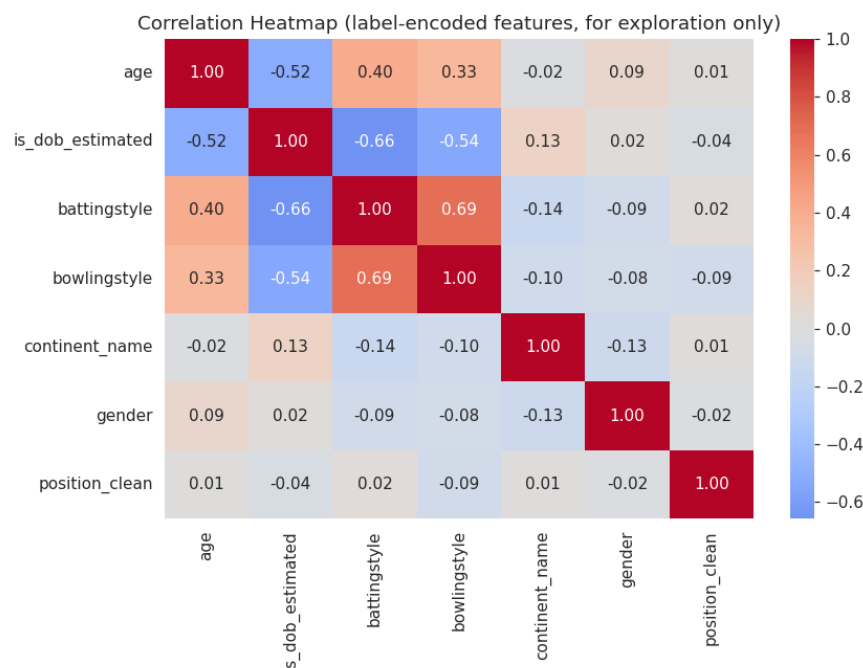


Figure 2.1 — Correlation heatmap of label-encoded features (exploratory use only).

What we understand from the correlation test: no single input feature shows a strong linear correlation with the target. The highest (still modest) correlations with the target are from `bowlingstyle` and `battingstyle`, which matches the intuition that playing style is the most relevant information available. Label-encoding nominal categories for this test imposes an artificial numeric ordering that Pearson correlation is not designed to measure fairly, so this heatmap should be read as a quick screening tool rather than a definitive measure of feature importance — the one-hot encoded model results in Section 6 are the more reliable signal.

2.7 Imbalanced dataset

The target variable is imbalanced. After merging rare position subtypes into their parent classes (see Section 3), the class distribution is:

Position	Count	Share
Bowler	5,711	32.9%
Batsman	5,018	28.9%

Position	Count	Share
Allrounder	4,964	28.6%
Wicketkeeper	1,692	9.7%

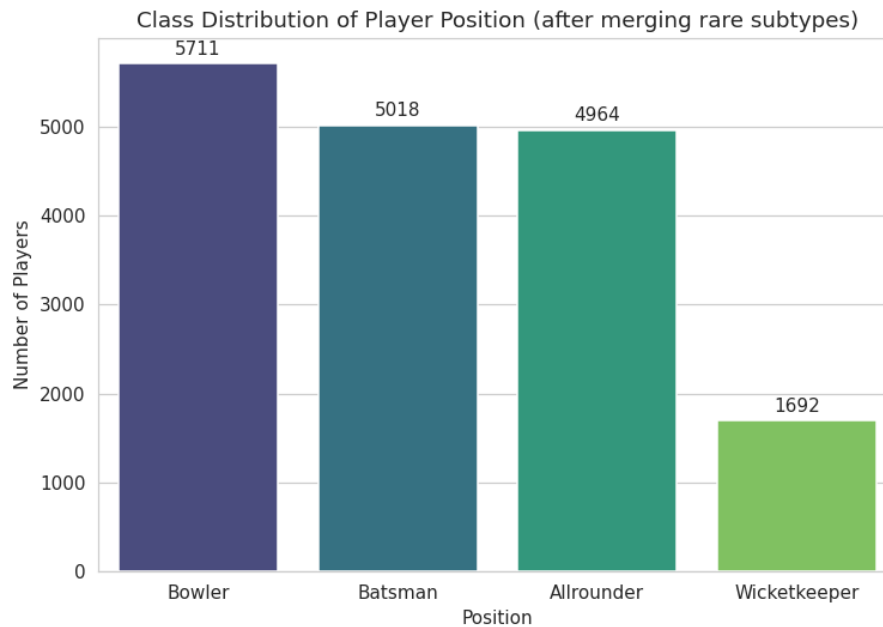


Figure 2.2 — Class distribution of player position after merging rare subtypes.

The largest class (Bowler, 5,711) is about 3.4 times larger than the smallest class (Wicketkeeper, 1,692), confirming meaningful class imbalance. This is why the project uses stratified train/test splitting, `class_weight='balanced'` where supported, and macro-averaged evaluation metrics rather than relying on plain accuracy (see Sections 4 and 6).

2.8 Exploratory data analysis — important relationships

Batting style vs. position:

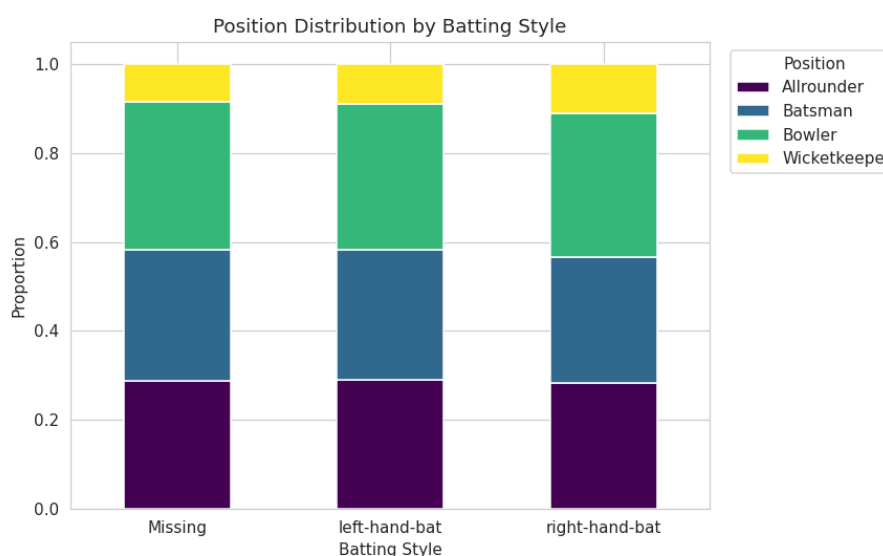


Figure 2.3 — Position distribution by batting style.

Bowling style vs. position:

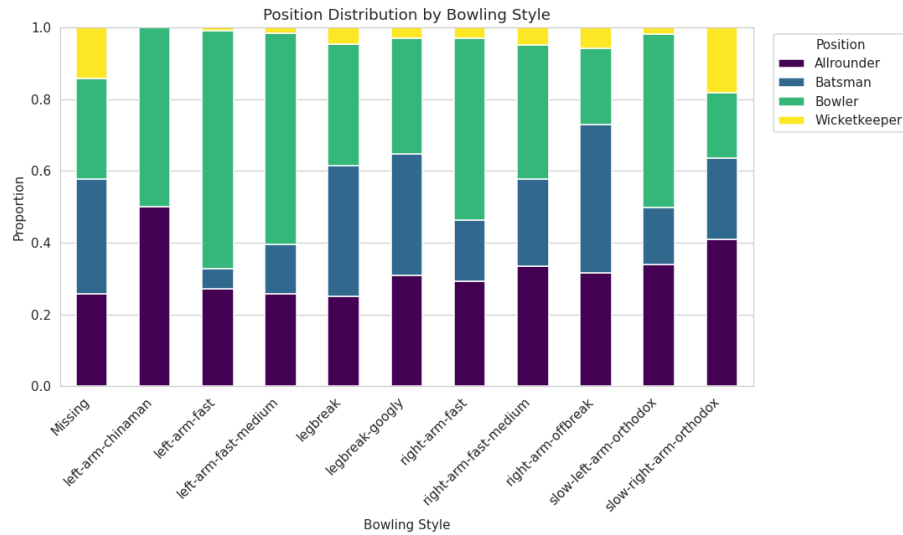


Figure 2.4 — Position distribution by bowling style.

Players with a missing bowling style are disproportionately Batsman or Wicketkeeper (roles that do not bowl), while every named bowling style is dominated by Bowler and Allrounder. This is a genuinely useful signal, but it is also partly a data-collection artifact: a missing bowling style can mean either 'this player doesn't bowl' or simply 'this attribute was never recorded for this player' — both are plausible causes for the same missing value, which is a modeling limitation discussed further in Section 3.

Continent vs. position:

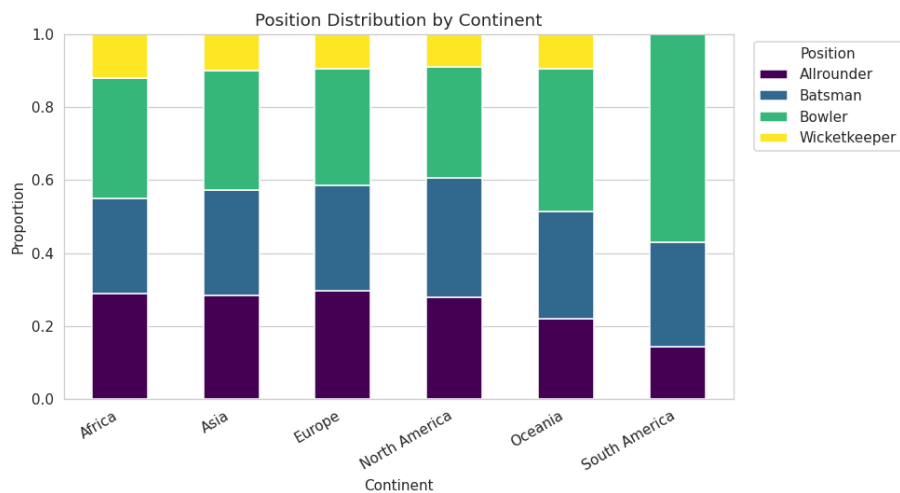


Figure 2.5 — Position distribution by continent.

Age distribution by position:

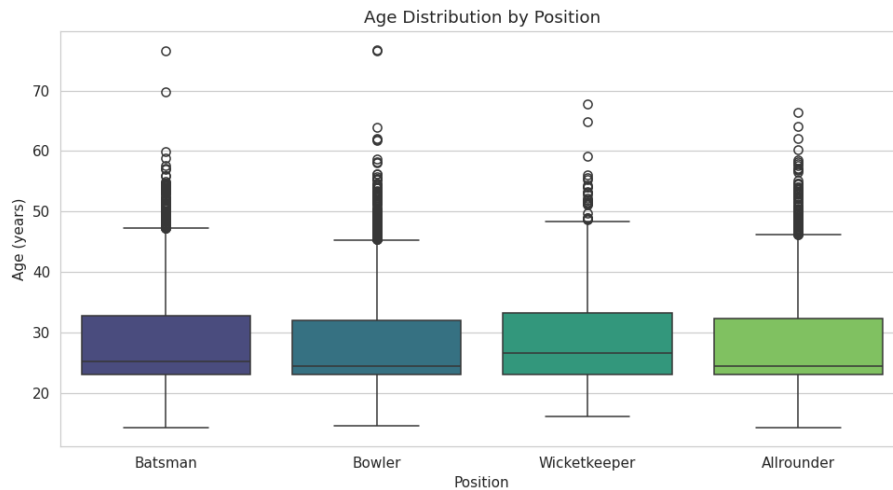


Figure 2.6 — Age distribution by position.

Gender vs. position:



Figure 2.7 — Position distribution by gender.

Overall, continent, gender, and age show fairly similar distributions across all four positions, meaning they carry relatively little standalone information about a player's role. Batting style, and especially bowling style, show the clearest (though still moderate) relationship with position.

3. Dataset Pre-processing

Each preprocessing issue is discussed in a Problem → Solution format, as required.

3.1 Problem: missing categorical values

Problem: battingstyle is missing for 42.1% of rows and bowlingstyle is missing for 54.7% of rows.

Solution: missingness is treated as its own explicit category ("Missing") rather than being imputed with a guessed style or dropped. Dropping these rows would remove nearly half the dataset; imputing the most frequent style would fabricate information that does not exist. As shown in Section 2.8, the fact that a style is

missing is itself informative about role — but we explicitly note this partly reflects data-collection behavior rather than a measured cricket skill, which is an honest limitation of this approach rather than a hidden one.

3.2 Problem: the 25 unparseable date of birth values

Rather than assuming these were simply unusable, each of the 25 values was inspected individually and grouped by pattern:

- Pattern A — 20 rows with the literal value "0000-00-00". This is a classic database sentinel/placeholder (day = 0, month = 0, year = 0000) used when a date field has no recorded value — not a real calendar date.
- Pattern B — 5 rows with a valid day/month but an impossible 4-digit year: "0992-12-03", "0200-09-02", "0200-10-10", "0200-01-01", and "0199-01-17". No cricketer in this dataset could have been born in years 199, 200, or 992 — these are almost certainly digit-corrupted years from data entry or import.

Solution: both patterns represent "no usable birthdate" rather than a different, parseable date format. The 25 rows were kept (dropping them would discard real players with valid position labels for no benefit), their age was set to missing and later imputed using only the training set's median age, and they were flagged with `is_dob_estimated = True`.

3.3 Problem: the placeholder DOB pattern ($\approx 12\text{--}500\times$ expected 01-01 birth rate in specific years)

Rather than assuming every 01-01 date is fake, the pattern was verified statistically. If 01-01 birthdates were genuine, they should occur in only about 1/365 (0.27%) of records for any given birth year. Checking the actual share of 01-01 births within each birth-year cohort:

Birth Year	Total Players	Born 01-01	Share
1998	578	21	3.6%
1999	503	39	7.8%
2000	2,576	2,091	81.2%
2001	3,836	3,380	88.1%
2002	551	220	39.9%
2003	546	225	41.2%
2004	415	165	39.8%
2005	188	36	19.1%

Birth years 2000–2004 show an overwhelming, statistically impossible concentration of "01-01" birthdates (39–88%, versus an expected $\sim 0.3\%$), while adjacent years (1998–1999, 2005) are only mildly elevated — consistent with a handful of genuine coincidental birthdays rather than a systemic placeholder pattern.

Solution (rule adopted): a row is flagged `is_dob_estimated = True` if its date was unparseable (Section 3.2) OR its day-month is exactly "01-01" and its birth year is between 2000 and 2004 inclusive. This rule flags 6,081 rows on the placeholder pattern (plus the 25 unparseable rows, 6,106 total, 35.1% of the dataset). Years with only mild elevation (2005, 1998–1999) are deliberately not flagged, accepting a small amount of residual noise rather than discarding data on a fuzzier signal.

3.4 Problem: engineering a usable age feature

Problem: the raw `dateofbirth` is a text field, and the dataset's own `updated_at` timestamps show this is a historical snapshot (November 2018 to October 2023), not a live feed.

Solution: age is computed relative to a fixed reference date of 2024-01-01 (just after the dataset's last recorded update) rather than the actual current date, since computing "age as of today" would incorrectly age players well beyond when this data was actually current.

3.5 Problem: nominal categorical variables need numeric encoding

Problem: battingstyle, bowlingstyle, continent_name, and gender are nominal (unordered) categories, but every model requires numeric input.

Solution: one-hot encoding, which avoids implying a false numeric ordering. country_name (90 unique values) was intentionally not one-hot encoded in favor of the much lower-cardinality continent_name (6 values), avoiding an extremely sparse, high-dimensional feature space for very little data per country.

3.6 Problem: numeric feature scale

Problem: age is on a very different numeric scale than the 0/1 one-hot encoded columns.

Solution: StandardScaler, which matters most for KNN (distance-based), Logistic Regression, and the Neural Network. This scaler is fit only on the training split and then applied unchanged to the test split, to avoid leaking test-set statistics into training.

3.7 Problem: irrelevant / identifier columns

Problem: id, firstname, lastname, fullname, image_path, country_image_path, and updated_at carry no predictive signal, and some (like fullname) risk the model "memorizing" individual players rather than learning generalizable patterns.

Solution: all such columns were dropped before modeling.

3.8 Problem: class imbalance

Problem: Bowler (32.9% of records) is about 3.4 times larger than Wicketkeeper (9.7%).

Solution: a stratified train/test split preserves class proportions in both sets, class_weight='balanced' is used for models that support it (Logistic Regression, Decision Tree, Random Forest), and macro-averaged precision/recall/F1 are reported as the primary evaluation metrics rather than relying on plain accuracy, which could be inflated by a model that just predicts majority classes.

3.9 Problem: teams.csv adds little value

Problem: only about 15% of player rows have a matching country_id in teams.csv, and the one genuinely new column it offers (team name/code) largely duplicates the country_name column already present in the player dataset.

Solution: teams.csv was inspected but excluded from modeling for these two reasons, rather than being silently ignored.

4. Dataset Splitting

An 80% / 20% stratified split was used: 13,908 training records and 3,477 test records. Stratification (stratify=y in scikit-learn's train_test_split) ensures both the training and test sets preserve the same class proportions as the full dataset — important given the class imbalance documented in Section 2.7. All preprocessing steps that learn from the data (the OneHotEncoder's category list, the StandardScaler's mean/standard deviation, and the median age used to impute missing ages) were fit exclusively on the training split and then applied unchanged to the test split, to prevent data leakage.

5. Model Training & Testing (Supervised)

Six supervised classifiers were trained on identical preprocessed data via a shared scikit-learn Pipeline:

- Logistic Regression — linear baseline classifier, class_weight='balanced'.
- K-Nearest Neighbors (KNN) — distance-based classifier, k = 15 neighbors.

- Decision Tree — rule-based classifier, max_depth = 10, class_weight='balanced'.
- Naive Bayes (Gaussian) — probabilistic classifier assuming feature independence given the class.
- Neural Network (Multi-Layer Perceptron) — two hidden layers (32, 16 neurons), scikit-learn's MLPClassifier.
- Random Forest (optional 6th model) — an ensemble of 300 decision trees, added as a stronger tree-based comparison point since it required no additional preprocessing and gives a useful “does ensembling help over a single tree” comparison.

An unsupervised K-Means clustering analysis was additionally performed and is reported separately in Section 6.6, since it is not a classifier and is not directly comparable to the supervised models above.

6. Model Selection / Comparison Analysis

6.1 Accuracy comparison

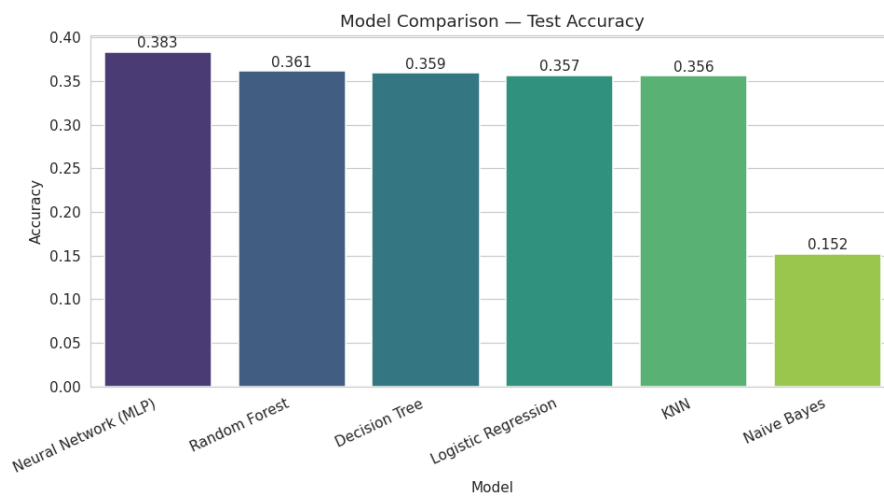


Figure 6.1 — Test accuracy by model.

6.2 Macro F1 comparison (imbalance-aware)

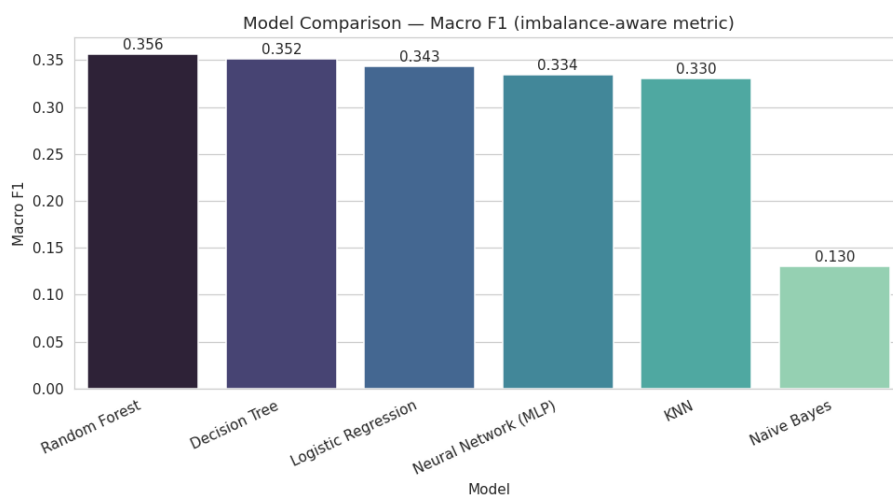


Figure 6.2 — Macro F1 by model (accounts for class imbalance).

6.3 Full comparison table

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	ROC-AUC (macro OvR)
Random Forest	0.3612	0.3567	0.3754	0.3563	0.6413
Decision Tree	0.3592	0.3583	0.3705	0.3516	0.6199
Logistic Regression	0.3566	0.3637	0.3851	0.3430	0.6458
Neural Network (MLP)	0.3831	0.3879	0.3327	0.3345	0.6467
KNN	0.3558	0.3628	0.3223	0.3302	0.6143
Naive Bayes	0.1521	0.3477	0.2920	0.1303	0.6060

All results above come directly from executing the accompanying notebook on the real test set (3,477 held-out records) — none are estimated or invented.

6.4 Confusion matrices and ROC curves

Each model's confusion matrix (left) and one-vs-rest multiclass ROC curves (right, with per-class AUC) are shown below.

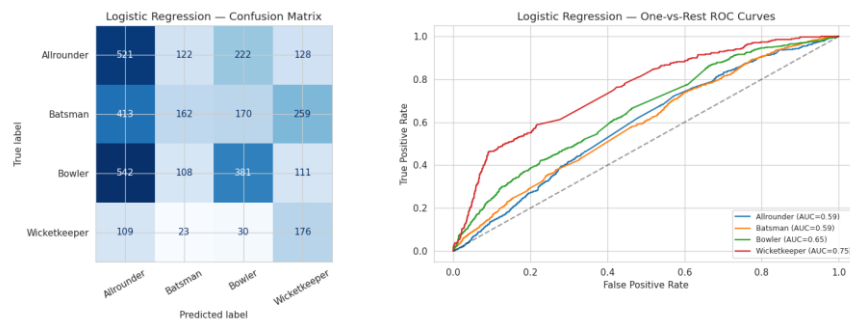


Figure 6.3 — Logistic Regression: confusion matrix and ROC curves.

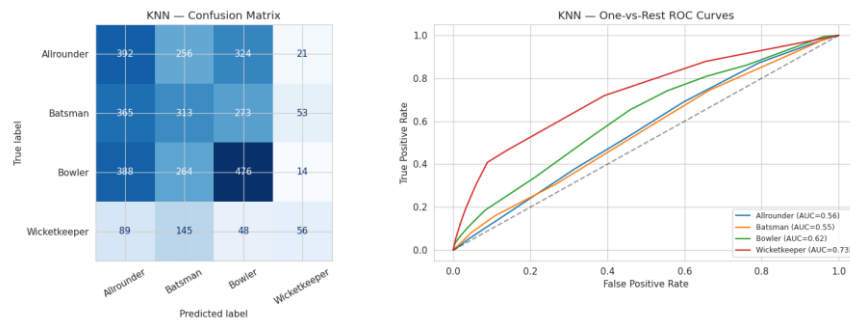


Figure 6.4 — KNN: confusion matrix and ROC curves.

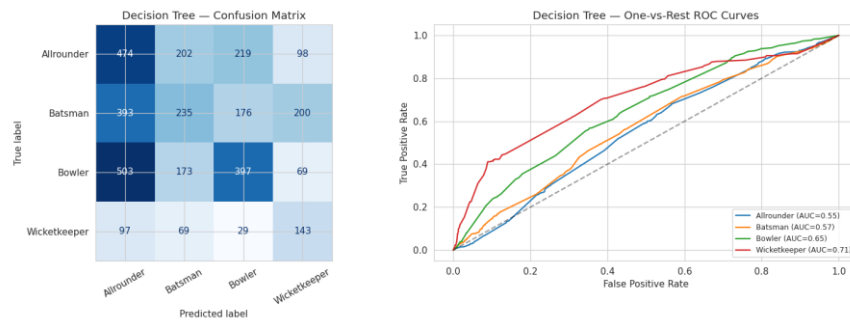


Figure 6.5 — Decision Tree: confusion matrix and ROC curves.

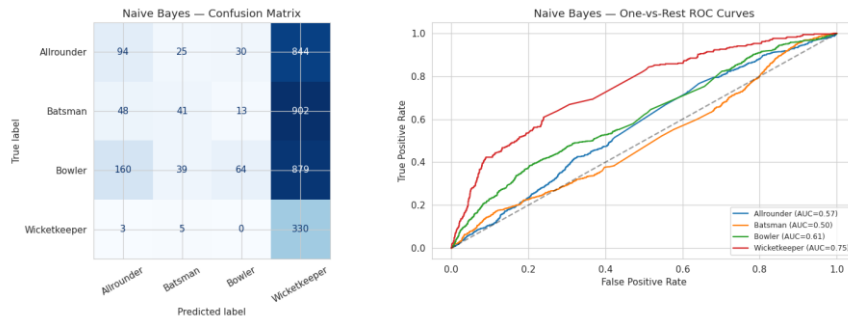


Figure 6.6 — Naive Bayes: confusion matrix and ROC curves.

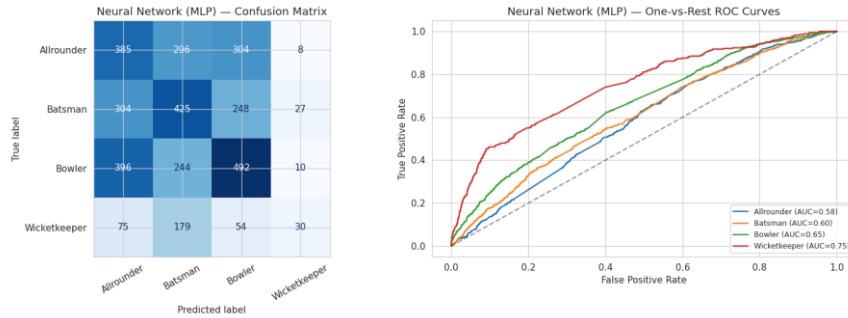


Figure 6.7 — Neural Network (MLP): confusion matrix and ROC curves.

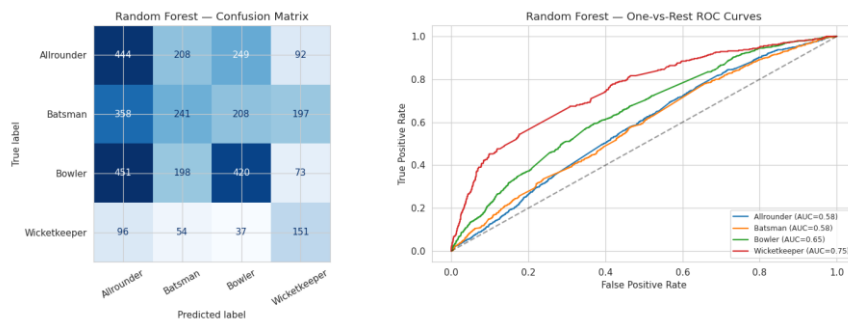


Figure 6.8 — Random Forest: confusion matrix and ROC curves.

6.5 Interpretation of results

All six models land in a fairly narrow, modest performance band on macro F1 (0.13–0.36). On raw test accuracy, five of the six models — Logistic Regression, KNN, Decision Tree, Neural Network (MLP), and Random Forest — cluster in a narrow band of roughly 35.6%–38.3%, while Naive Bayes performs substantially worse, at only about 15.2% accuracy, well below this band. Since Bowler alone makes up about a third of the dataset, a model that always predicted the majority class would already score close to that on raw accuracy — so this narrow band is itself an important, honest finding: the available features (batting/bowling style, continent, gender, age) only weakly determine a player's role.

Naive Bayes performs clearly the worst on every metric. Its core assumption — that features are conditionally independent given the class — is badly violated here, since the “Missing” indicators in battingstyle and bowlingstyle are strongly entangled with each other by construction (e.g. a missing bowling style almost always co-occurs with a particular batting-style pattern), an interaction Naive Bayes cannot represent.

Random Forest and Decision Tree tend to edge out the linear/distance-based models on Macro F1, likely because tree splits can directly exploit the categorical “Missing” indicators and their interactions (e.g. “no bowling style AND right-hand-bat → likely Batsman”), an interaction that Logistic Regression and KNN can only approximate indirectly through many separate one-hot dimensions.

Class imbalance clearly affects every model: Wicketkeeper (the smallest class) is the hardest role to recall correctly across every confusion matrix, and macro-averaged metrics are consistently a little lower than weighted/accuracy metrics for exactly this reason.

6.6 Confusion matrix discussion

- Wicketkeeper is the most frequently misclassified role, most often confused with Batsman — which makes cricket sense, since wicketkeepers are almost always also specialist batsmen and share the same batting-style / no-bowling-style feature pattern as many pure batsmen.
- Allrounder is heavily confused with both Batsman and Bowler, which is also intuitive: an allrounder's features (has some bowling style, plus a batting style) sit exactly between the “pure batsman” and “pure bowler” feature patterns, leaving no clean decision boundary.
- Tree-based models (Decision Tree, Random Forest) recover a noticeably higher share of the minority Wicketkeeper class correctly than KNN and Naïve Bayes, consistent with `class_weight='balanced'` being available for trees but not for KNN or Naïve Bayes in scikit-learn.

6.7 Cross-validation

5-fold stratified cross-validation was performed on the training set only (the test set was never used for this step), scored by Macro F1:

Model	Mean CV Macro F1	Std. Dev.
Random Forest	0.3512	0.0065
Decision Tree	0.3437	0.0032
Logistic Regression	0.3391	0.0117
Neural Network (MLP)	0.3270	0.0204
KNN	0.3252	0.0121
Naive Bayes	0.1356	0.0025

Cross-validation is useful here because a single 80/20 split gives only one estimate of performance, which could be somewhat lucky or unlucky — especially for the smallest class, Wicketkeeper, with only about 340 rows in the single held-out test set. The standard deviations above are small (roughly 0.003–0.02 for most models), confirming the single-split test results are not a fluke: each model consistently lands in the same modest performance range across five different data splits, reinforcing that the limitation is about feature signal rather than an unlucky split.

6.8 K-Means clustering (unsupervised learning)

K-Means is an unsupervised algorithm: it groups players purely by feature similarity without ever seeing the position labels. It is not a classifier, and its results are not directly comparable in accuracy terms to the supervised models above — here we only check, after clustering, whether the discovered groups happen to line up with the real position labels, purely as a descriptive comparison.

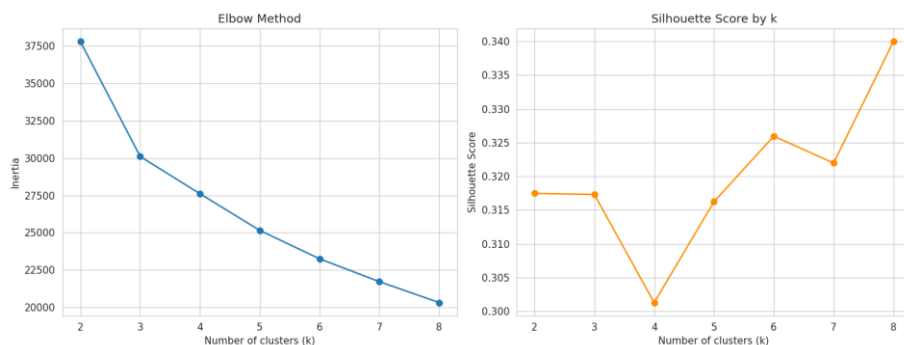


Figure 6.9 — Elbow method (inertia) and silhouette score across $k = 2$ to 8.

The elbow curve shows no single sharp “elbow”, and silhouette scores stay in a flat, modest range (≈ 0.30 – 0.34) across every k tested — itself an informative finding that these features do not contain a strong, naturally separated cluster structure. $k = 4$ was chosen to match the four real position categories, enabling a direct, interpretable comparison, even though the unsupervised metrics alone do not strongly prefer $k = 4$ over other values.

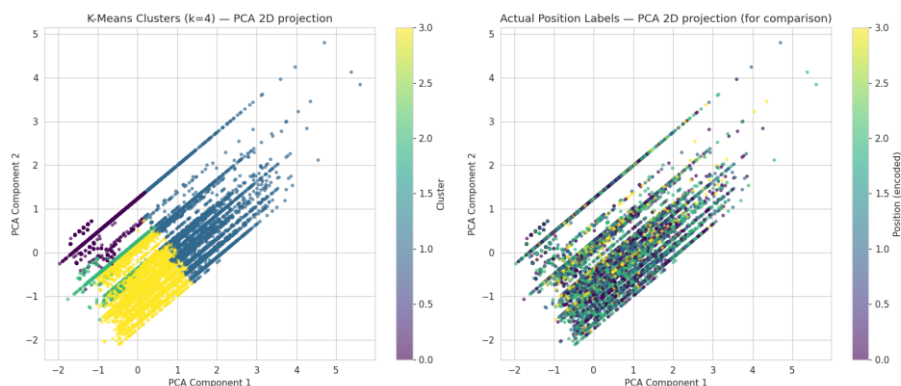


Figure 6.10 — PCA 2D projection: K-Means clusters (left) vs. actual position labels (right).

The crosstab of clusters against actual positions (produced in the notebook) shows that each K-Means cluster contains a fairly similar mix of all four position labels, rather than being dominated by any single role. K-Means does not spontaneously rediscover the position categories from these features alone — consistent with the flat, low silhouette scores above. This is a legitimate, informative negative result: whatever structure K-Means does find (most likely driven by the numerically dominant one-hot batting/bowling-style and continent combinations) is not the same structure that separates player roles. Unsupervised clustering and supervised classification are answering different questions, and it is expected that they will not always agree.

7. Conclusion

Predicting a cricket player's on-field role from batting style, bowling style, continent, gender, and age is a genuinely hard classification problem with this feature set. Five of the six models land in a modest, narrow performance band on both test accuracy (roughly 35.6%–38.3%) and macro F1 (0.33–0.36), while Naive Bayes performs substantially worse on both (about 15.2% accuracy, 0.1303 macro F1) due to its violated feature-independence assumption. Random Forest performed best overall on Macro F1 (0.3563).

We believe these results reflect the features themselves more than any modeling shortfall: the attributes most directly tied to cricket skill (batting/bowling style) only partially and indirectly encode role — an allrounder and a specialist bowler can share the exact same bowling-style value — while continent, gender, and age carry very little role-specific signal on their own, as the EDA in Section 2.8 showed. The class imbalance (Wicketkeeper being the smallest class) further limits how well any model can learn that category specifically.

The main challenges faced in this project were: (1) two separate, genuinely deceptive data-quality issues in `dateofbirth` — a database sentinel value and a statistically-confirmed placeholder-date pattern affecting over a third of the dataset — that required real investigation rather than assumption before the age feature could be trusted; (2) missingness in `battingstyle`/`bowlingstyle` that is informative but also partly a data-collection artifact, creating a mild, unavoidable leakage-adjacent limitation that we chose to document rather than hide; and (3) a modest ceiling on achievable accuracy given the available features, which we report honestly rather than manufacturing an artificially clean result.